Práctica Inteligencia Artificial: Control de un reactor nuclear con incertidumbre

Inteligencia Artificial

Grado en Ingeniería Informática
Doble Grado en Ingeniería Informática y Administración de Empresas
Grado en Inteligencia Artificial

Curso académico 2025-2026

Índice

1 Enunciado **2**

2 Tareas **5**
2.1 Modelado del MDP 5
2.2 Generación del MDP por iteración de control 5
2.3 Integración del bucle de control 6
2.4 Implementación de métricas de evaluación 7
2.5 Experimentación y evaluación con distintos reactores 8

3 Código proporcionado **8**

4 Evaluación **9**
4.1 Modelado del MDP 9
4.2 Generación del MDP por iteración de control 10
4.3 Integración del bucle de control 10
4.4 Implementación de métricas de evaluación 10
4.5 Experimentación 10
4.6 Memoria 10

5 Normas **11**

6 Uso de IA generativa **11**

1. Enunciado

La energía nuclear supone una de las fuentes de energía no-renovable más eficientes y limpias de las que dispone el ser humano en la actualidad, siendo de gran interés por su capacidad de generación de grandes cantidades de energía eléctrica a partir de poco combustible nuclear. En esencia, su funcionamiento es similar al de cualquier central térmica convencional: calentar el agua hasta generar vapor de alta presión que haga girar una turbina a gran velocidad, la cual, mediante el fenómeno de inducción electromagnética, genere la deseada corriente eléctrica (véase la Figura 1). Las centrales nucleares emplean reactores nucleares donde, en vez de realizar la quema de combustible fósil, se realiza un proceso de fisión controlada del material enriquecido (Uranio-235 o U^{235}, mediante pastillas de óxido de uranio UO_2) que libera una gran cantidad de calor, el cual calienta un primer circuito de agua (altamente contaminada) que a su vez calienta un segundo circuito cuya finalidad es hacer mover la turbina.

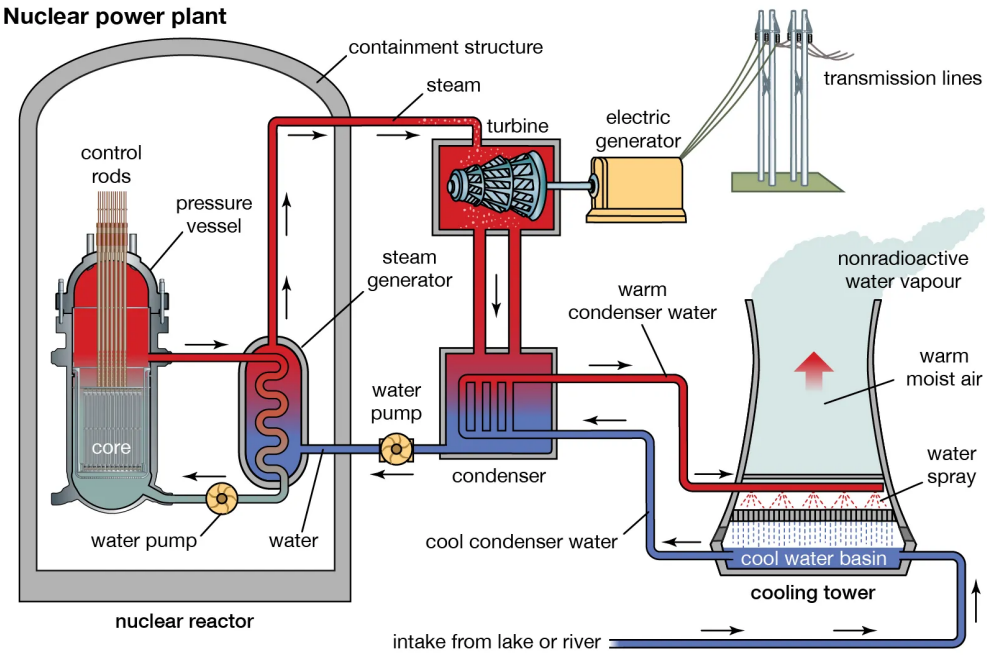

Figura 1: Esquema del funcionamiento de una central nuclear.

Para controlar cuánto calor produce el reactor (y por tanto: cuánta energía se genera) se emplea un mecanismo conocido como *barras de control*, un conjunto de barras compuestas habitualmente de boro, elemento conocido por sus capacidades de absorción neutrónica, que permite limitar el número de neutrones que participan en la reacción en cadena de la fisión, controlando así la temperatura del reactor. Su funcionamiento podría verse asimilado al de una "barrera" que frena la reacción, de modo que, cuantas más barras estén insertadas en el núcleo del reactor, menor será la reactividad y en consecuencia menor será la energía producida (y viceversa).

En definitiva, sabemos que la potencia $P = \frac{dW}{dt}$ del reactor dependerá de qué porcentaje de las barras de control esté insertado. Formalizando el conocimiento anterior de manera simplificada, tenemos las siguientes características relevantes en el diseño del núcleo de un reactor:

- P: Potencia térmica (W).
- Σ_f: Sección eficaz macroscópica de fisión (cm^{-1}).
- ϕ: Flujo neutrónico (neutrones/($cm^2 \cdot s$)).

- V: Volumen del núcleo del reactor (cm^3).
- E_f: Energía liberada por fisión (J).

Todas estas características se interrelacionan en la Ecuación 1, que nos permite conocer la potencia máxima a la que es capaz de operar un determinado reactor.

$$P_{\text{máx}} = \Sigma_f \cdot \phi \cdot V \cdot E_f \tag{1}$$

Gracias a la posibilidad de conocer la potencia máxima teórica de un reactor, podemos modelar una ecuación que nos permitirá conocer a qué potencia operará el reactor en función del porcentaje de barras de control insertadas en el núcleo (Ecuación 2):

$$P(B) = P_{\text{máx}} \cdot e^{-kB} \mid k = -ln\left(\frac{10^{-6}}{P_{\text{máx}}}\right) \tag{2}$$

Donde B es el porcentaje de barras de control insertadas $B \in [0, 1]$ y donde k es una constante que toma su correspondiente valor en función de $P_{\text{máx}}$. Para comprender mejor este modelo dinámico, sustituyamos por B el valor 100 % (es decir: 1), simulando que todas las barras de control han sido insertadas. Lo que se obtiene es:

$$P(B = 1) = P_{\text{máx}} \cdot e^{-k \cdot 1} = P_{\text{máx}} \cdot e^{-k} = P_{\text{máx}} \cdot e^{ln\left(\frac{10^{-6}}{P_{\text{máx}}}\right)} = 10^{-6} \approx 0$$

Es decir, si se insertan todas las barras de control, se obtiene un valor de potencia muy cercano a cero. Por el lado contrario, veamos qué ocurre cuando se sustituye $B = 0$:

$$P(B = 0) = P_{\text{máx}} \cdot e^{-k \cdot 0} = P_{\text{máx}} \cdot e^{0} = P_{\text{máx}} \cdot 1 = P_{\text{máx}}$$

Como era de esperar, el reactor opera a máxima potencia cuando no hay barras de control insertadas, cumpliendo con el comportamiento esperado por el modelo dinámico.

Hasta este punto todo parece correcto y aparentemente disponemos de un modelo dinámico analítico (exacto) que nos permite controlar a la perfección la potencia a la que opera nuestro reactor. La realidad es que el proceso de fisión nuclear es increíblemente más complejo y dispone de una serie de detalles y características imposibles de controlar, predecir o directamente medir con precisión, lo cual hace que el manejo de los reactores tenga un considerable grado de incertidumbre que los hace especialmente difíciles de controlar, algo crítico teniendo en cuenta el riesgo que acarrean los posibles accidentes nucleares.

La realidad es que en los reactores, cuando se pretende subir o bajar la potencia a la que operan, esta acción no siempre responde de forma determinista. Por ejemplo: deseamos bajar la potencia de operación una unidad en la escala de producción y, por un envenamiento puntual de xenón (muy común en reactores), la potencia baja dos unidades en vez de una. Para llevar a cabo el control sometido a este grado de incertidumbre, los reactores son estudiados sistemáticamente con el fin de conocer las probabilidades asociadas a las distintas acciones que se pueden realizar.

En el problema propuesto por la presente práctica, una central nuclear española ha decidido recurrir al uso de MDP's (*Procesos de Decisión de Markov*) como técnica central para poder gestionar el control con incertidumbre de los reactores nucleares. La idea fundamental es que un reactor nuclear tiene la potencia estratificada en 100 niveles discretos (porcentajes de potencia a la que opera el reactor), que van del 0 al 99. De este modo, el primer nivel sería el $0\,\% - 1\,\%$, el segundo sería el $1\,\% - 2\,\%$, y así hasta llegar al $99\,\% - 100\,\%$. Cualquier nivel de potencial real, es automáticamente discretizado en estos intervalos, por ejemplo: la potencia del 33.88 % se encontraría discretizada en el nivel 33, que ocupa del 33 % al 34 % de potencia. Para una mejor comprensión de este concepto, véase la Figura 2.

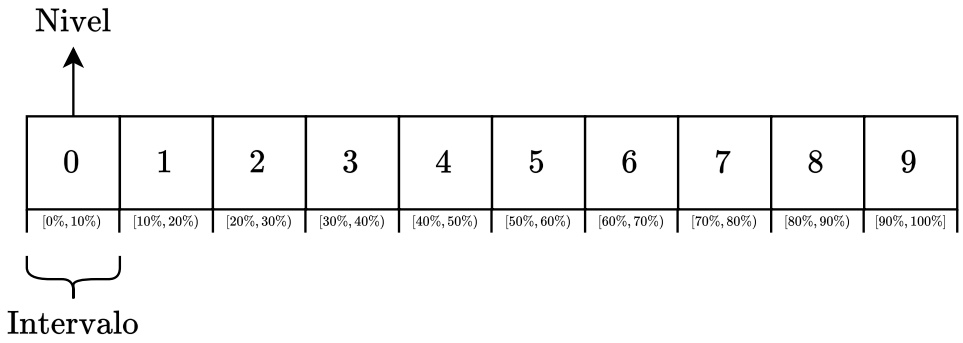

Figura 2: Ejemplo de niveles de potencia y sus correspondientes intervalos en una simplificación de 10 niveles.

Por otro lado, las tres únicas operaciones que se pueden realizar en la tarea de control de un reactor son:

1. Decrementar: Denominado como la acción *decrease* (d), tiene como objetivo reducir la potencia del reactor en un nivel. Con cierta probabilidad, a veces la potencia no se verá reducida y en otros casos la potencia se verá excesivamente reducida en dos niveles.

2. Mantener: Denominado como la acción *maintain* (m), tiene como objetivo mantener la potencia del reactor en un nivel concreto. Con cierta probabilidad, a veces la potencia se mantendrá en el nivel deseado, pero otras veces se incrementará en un nivel, o se decrementará en uno.

3. Incrementar: Denominado como la acción *increase* (i), tiene como objetivo incrementar la potencia del reactor en un nivel. Con cierta probabilidad, a veces la potencia no se verá incrementada y en otros casos la potencia se verá excesivamente incrementada en dos niveles.

Resumiendo los posibles resultados de las acciones en el reactor, tenemos:

Acción	-2	-1	0
d	`No deseado`	`Deseado`	`No deseado`

Cuadro 1: Posibles resultados asociados a la acción *decrease* (d).

Acción	-1	0	+1
m	`No deseado`	`Deseado`	`No deseado`

Cuadro 2: Posibles resultados asociados a la acción *maintain* (m).

Acción	0	+1	+2
i	`No deseado`	`Deseado`	`No deseado`

Cuadro 3: Posibles resultados asociados a la acción *increase* (i).

En conclusión, el modelo dinámico nos permitirá conocer a qué potencia operará nuestro reactor en función del porcentaje de barras de control insertadas, pero la incertidumbre en las tres acciones de control posibles tendrá que ser manejada mediante un MDP que permita aprender, para cada estado, la política óptima (mejor acción) que debe llevarse a cabo. En los siguientes apartados se detallarán los aspectos técnicos relacionados con el trabajo que se espera del alumno.

2. Tareas

Para lograr los objetivos propuestos, la práctica queda desglosada en la siguiente secuencia de tareas que deben ser completadas a modo de ejercicios separados pero relacionados entre sí.

2.1. Modelado del MDP

Para este apartado se pide:

1. Modelar matemáticamente el *Proceso de Decisión de Markov* (MDP), empleando la definición fundamental estudiada en clase. Es muy importante dar todo el detalle necesario que permita definir los estados, las acciones, las transiciones y los costes del MDP.

2. Calcular cuántos estados y acciones hay en el MDP, así como el número de dimensiones tendrán las matrices de transición $\mathcal{P}$ y de costes $\mathcal{C}$.

3. Realizar una representación visual de tipo grafo que describa la topología (red) del MDP.

2.2. Generación del MDP por iteración de control

La manera en la que se espera que el sistema de control emplee un MDP es la siguiente:

- La central recibe una curva de demanda energética (potencia a la que la red necesita que el reactor funcione del 0 % al 100 %), compuesta por una sucesión de puntos de demanda.

- Seleccionemos el punto actual de demanda y calculemos la diferencia entre dicha demanda y la potencia que suministra cada uno de los niveles (escogiendo el límite inferior de cada intervalo).

- Por ejemplo, si la demanda $d_t = 0.33$, la diferencia con el nivel 10 sería $\Delta_t^{(10)} = 0.33 - 0.1 = 0.23$. Para el nivel 50, la diferencia sería $\Delta_t^{(50)} = 0.33 - 0.5 = -0.17$.

- Si se calcula el valor absoluto de dichas diferencias, se obtienen las distancias entre la demanda actual y los distintos niveles, sirviendo éstas como los costes que empleará el MDP.

- Como requisito adicional, se deberán multiplicar $\times 2$ las distancias (costes) asociadas a aquellas acciones que se alejan del objetivo. Para entender esto, veamos el siguiente ejemplo: la demanda actual es $d_t = 0.33$ y pretendo calcular el coste $C(s, a, s')$ asociado a transicionar del nivel 50 al 51 de potencia mediante la acción *increase*. La distancia originalmente sería $|\Delta_t^{(51)}| = |0.33 - 0.51| = 0.18$ de modo que el nivel destino 51 (cuya potencia es de 0.51) estaba por encima de 0.33 y haber tomado la acción *increase* implica alejarse activamente de la potencia objetivo establecida por la demanda actual, por ello: $C(s, a, s') = 2 \cdot |\Delta_t^{(51)}| = 0.36$. Del mismo modo ocurre para la acción *decrease*, donde decrementar el nivel de potencia cuando la demanda está por encima también debe ser penalizado.

- Este proceso implica recalcular la matriz de costes $\mathcal{C}$ y debe repetirse iterativamente por cada iteración de control, que es equivalente a repetirse por cada punto en la curva de demanda, ajustando los costes del MDP para que se penalicen aquellas distancias que se asocian a transiciones contradictorias con el objetivo de alcanzar el nivel de demanda actual (véase la Figura 3).

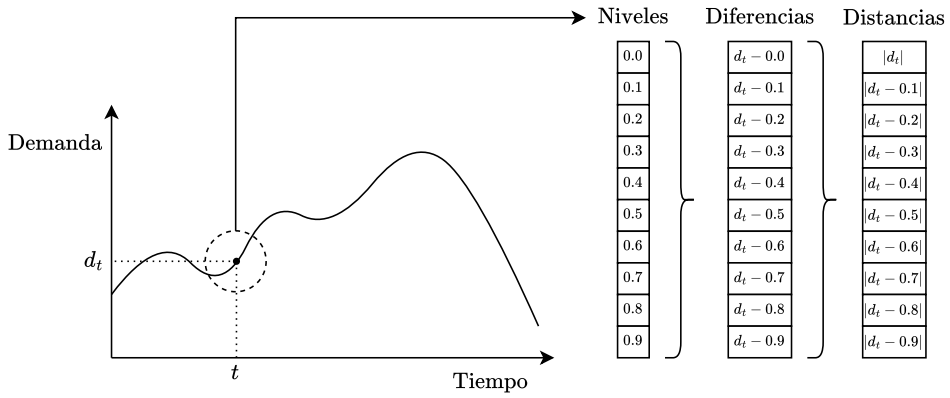

Figura 3: Ejemplo del cálculo de distancias / costes a partir del punto de demanda actual t.

Para completar este apartado, se pide:

1. Implementar una función que permita generar la matriz de probabilidades (transición) $\mathcal{P}$ a partir de las probabilidades asociadas a las acciones definidas para un reactor concreto. Por cada reactor se dispondrá de 3 tablas de probabilidad (siguiendo el esquema de las tablas del apartado anterior) que establecerán la casuística de transición vinculada a cada una de las acciones. Es importante tener en cuenta que las probabilidades de transición han de ser iguales para cualquier estado y que nunca se podrá lograr una potencia inferior al 0 % o superior al 100 %. Nota: la matriz $\mathcal{P}$ solo debe ser calculada una vez, ya que se mantiene constante a lo largo de las distintas iteraciones.

2. Implementar una función que permita generar la matriz de costes $\mathcal{C}$ del MDP. Dicha matriz se debe calcular siguiendo las especificaciones anteriores, teniendo en cuenta que se debe generar una matriz por iteración (por tanto habrá un MDP por iteración) y que se deben penalizar las acciones que sean contradictorias con el objetivo de convergencia hacia el valor actual de demanda.

3. Integrar la generación de la matriz $\mathcal{P}$ y de la matriz $\mathcal{C}$ en una única función que resuelva una iteración de control, es decir, que resuelva qué acción (d, m o i) se debe tomar dado un nivel de demanda actual d_t. Dicha función será la que construya el MDP a partir de ambas matrices y la que resolverá mediante el algoritmo de *Iteración de Valores* cuál será la política óptima a ejecutar en cada estado. En base al nivel de potencia previo al instante t sobre el que se está calculando la demanda, se determinará en qué estado del MDP se encuentra el sistema para que la función devuelva exclusivamente la acción óptima en dicho estado. Para realizar esto, los alumnos deberán emplear la biblioteca de Python `pymdptoolbox` que implementa la definición de MDP's y la resolución de los mismos mediante distintos algoritmos, entre ellos, el de *Iteración de Valores*. Para dominar el uso de la biblioteca, el alumno es referido a la documentación oficial.

2.3. Integración del bucle de control

Habiendo implementado la función que ejecuta una iteración de control (último punto del apartado anterior) el sistema ya dispone de la respuesta del sistema de control para un nivel de demanda dado. Sin embargo, esto debe replicarse por cada puto en la curva que demanda que sirve como input del sistema. La ejecución de las iteraciones de control a lo largo de toda la curva de demanda energética lo denominaremos como bucle de control.

Para completar este apartado, se pide:

1. Implementar la función que realiza el bucle de control para todos los puntos que se reciben en la curva de demanda. La función invocará al método (previamente implementado) que resuelve una iteración

de control y que devuelve la acción que se debe realizar en el estado actual. La idea es que el sistema debe imitar el carácter estocástico del reactor, de modo que si la iteración de control determinó que se debía ejecutar la acción m en el instante t, el estado al que se transicione (-1, 0 o $+1$) debe ser escogido aleatoriamente siguiendo precisamente las probabilidades de transición definidas para el reactor. Para lograr que las respuestas del sistema transicionen probabilísticamente hacia los correspondientes estados se recomienda emplear la biblioteca de Python `Numpy` (concretamente el módulo `numpy.random`).

2. La salida de la función que implementa el bucle de control deberá ser el histórico de respuesta del reactor a la demanda, es decir, una serie temporal (recomendable almacenarla en un `Numpy Array`) con los distintos niveles de potencia (entre 0 y 1) con los que ha ido respondiendo el reactor a la demanda.

2.4. Implementación de métricas de evaluación

Habiendo completado el sistema de control del reactor, adicionalmente se le pide al alumno que implemente cuatro métricas que evalúan la calidad de una regresión, ya que nuestro problema consiste en que la curva de respuesta del reactor se asemeje lo máximo posible a la curva de demanda de potencia. Para ello, las cuatro métricas a implementar son:

1. Error Absoluto Medio: Calcula el promedio de la suma de los errores absolutos (diferencias entre el valor "real" = demanda y el valor "predicho" = respuesta). Su fórmula es:

$$\mathrm{MAE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

2. Error Cuadrático Medio: Calcula el promedio de la suma de los errores al cuadrado (diferencias entre el valor "real" = demanda y el valor "predicho" = respuesta). Su fórmula es:

$$\mathrm{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

3. Coeficiente R^2: Mide la calidad de la regresión comparándola con lo que un estimador de tipo media realizaría. Cuanto más cercano a 1 sea su valor, mejor será el regresor. Si por el contrario se obtienen valores ≤ 0, el coeficiente indica que el regresor es igual de eficaz o incluso peor que una media. Su fórmula es:

$$R^2(\mathbf{y}, \hat{\mathbf{y}}) = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

4. Coeficiente de Correlación de Pearson: Mide el grado de relación (positiva o negativa) entre dos variables. Devuelve valores acotados por el intervalo $[-1, 1]$. Si la regresión es perfecta, la correlación de Pearson es igual a 1. La restricción fundamental es que asume siempre correlaciones lineales. Su fórmula es:

$$Corr(\mathbf{y}, \hat{\mathbf{y}}) = \frac{Cov(\mathbf{y}, \hat{\mathbf{y}})}{\sigma(\mathbf{y})\sigma(\hat{\mathbf{y}})}$$

Dichas métricas de evaluación de la regresión serán empleadas para cuantificar y poder comparar cómo de bien han respondido los distintos reactores y sus correspondientes sistemas de control basados en MDP's a las curvas de demanda. Para la implementación, se recomienda usar `Numpy` aunque está permitido el uso de bibliotecas que faciliten estas métricas como `scikit-learn` o `Scipy`. Los conceptos de regresión y sus correspondientes métricas no son el objeto principal de la presente práctica, aunque sí son de gran utilidad y cobrarán sentido en futuras asignaturas como *Aprendizaje Automático*, *Redes de Neuronas Artificiales* y *Análisis de Datos*.

2.5. Experimentación y evaluación con distintos reactores

Se proporciona un conjunto de reactores nucleares definidos en ficheros de tipo JSON con distintas características físicas y con distribuciones de probabilidad variadas. La idea es que los alumnos prueben, como requisito necesario pero no suficiente, todos los reactores que se les proporcionan y, una vez hayan obtenido los correspondientes resultados (a partir de las gráficas que genera el sistema y de las métricas de calidad de la regresión previamente implementadas) sean capaces de agregar nuevos reactores ficticios con distintas características físicas, pero sobre todo, con distintas distribuciones de probabilidad, permitiendo probar casos extremos, reactores ideales, peligrosos, etc.

Es importante comprender que generar reactores con distribuciones de probabilidad aleatorias simplemente aumenta el grueso experimental pero esto no aporta valor a un sistema que ya ha sido validado como funcional a partir los reactores proporcionados por el enunciado. Por ello, es conveniente que los alumnos diseñen nuevas distribuciones de probabilidad que, como se ha mencionado previamente, permitan explorar los límites del sistema de control y permitan generar resultados que merezcan ser discutidos posteriormente en la memoria. No hay un número mínimo ni máximo de experimentos que se deban realizar, pero sí es conveniente que la experimentación sea lo suficientemente variada como para probar todos los posibles "tipos" de escenario.

3. Código proporcionado

Se proporciona un código base escrito en Python (lenguaje sobre el que se desarrollará la implementación del sistema) a partir del cual el alumno debe trabajar para implementar el sistema de control con MDP's. El objetivo por parte del alumno consiste en completar el código de ciertas funciones sin implementar que son clave en el sistema. Dichas funciones estarán sin programar aunque sí declaradas, para que el alumno pueda identificarlas fácilmente. Por otro lado, funcionalidades básicas para la generación de gráficas, así como la generación de curvas de demanda aleatorias, también estarán proporcionadas por el código base y los alumnos no deberán modificar estos módulos del sistema.

Además de las funciones a completar por parte del alumno, si éste estima oportuno implementar funciones adicionales con finalidades auxiliares, para refactorizar código o para facilitar la comprensión del sistema, le está totalmente permitido hacerlo. Se recuerda que es una buena práctica de programación el comentar todo lo que se pueda (sin caer en el exceso), para facilitar la comprensión del código tanto por parte de los alumnos como por parte del profesor. No obstante, para evitar potenciales errores al programar, se deberán tener muy en cuenta los tipos de datos de entrada / salida que reciben / devuelven los métodos a completar, para evitar fallos de coherencia con el resto del sistema que ya está implementado.

Se insta activamente a los alumnos a emplear la biblioteca `Numpy` de Python para las operaciones con *arrays*, que permitirán obtener un rendimiento muy superior al de las listas nativas de Python, además de facilitar la implementación de funciones que operan con matrices y/o vectores. Se trata de una biblioteca muy popular y ampliamente usada que todo programador en Python debería conocer y manejar.

Como se ha comentado previamente en el presente documento, junto con el código base se aportan ficheros `JSON` que contienen la configuración de distintos reactores que deberán ser probados por los alumnos para comprobar el correcto funcionamiento de sus sistemas de control. Se recuerda que el alumno debe ampliar esta batería de pruebas incluyendo escenarios que prueben situaciones donde se compruebe la robustez del sistema. Cada uno de estos ficheros JSON posee siguiente estructura:

```
{
    "model": "RBMK",                          // Modelo del reactor
    "effective_section": 17.6,                // Sección eficaz macroscópica
    "neutron_flux": 5e13,                     // Flujo neutrónico
    "core_volume": 9.42e6,                    // Volumen del núcleo del reactor
    "fision_energy": 3.2e-11,                 // Energía liberada por fisión
    "probabilities": {
        "decrease": [0.55, 0.20, 0.25],       // -2, -1, 0
        "maintain": [0.95, 0.025, 0.025],     // -1, 0, +1
        "increase": [0.65, 0.25, 0.1]         // 0, +1, +2
    }
}
```

Donde se destaca la importancia de reconocer y respetar el orden en el que se ubican las probabilidades asociadas a cada acción del reactor (especificado en los comentarios agregados al JSON de ejemplo). Partiendo de esta estructura, el alumno tendrá que replicarla para agregar los reactores que estime relevantes. Finalmente, es importante destacar que el IDE de desarrollo a utilizar durante la práctica será *Visual Studio Code*, ya que el código base contiene un fichero `launch.json` que permite ejecutar en modo *debug* el sistema, automatizando el paso de argumentos al programa y facilitando la depuración de errores. Se recomienda a los alumnos que analicen este fichero `launch.json` y repliquen la estructura que en ellos se encuentra para poder automatizar la ejecución de distintos experimentos sin tener que variar manualmente los argumentos que se le pasan inicialmente al programa. De manera adicional, se permitirá el uso de *PyCharm* como IDE de desarrollo alternativo.

4. Evaluación

En la siguiente secuencia de apartados se detallan las distintas puntuaciones asociadas a cada una de las tareas que el alumno deberá realizar. En total el **bloque práctico** ocupa **7 ptos.** de la práctica y la **memoria** a realizar representa los **3 ptos.** restantes.

4.1. Modelado del MDP

Este apartado suma un total de **1 pto**. Su desglose es:

- Definir formalmente (matemáticamente) el MDP que se empleará en cada iteración de control. (**0.7 ptos.**)
- Calcular el número de estados y de acciones, así como las dimensiones de las matrices de transición y de costes. (**0.2 ptos.**)
- Realizar un grafo que represente visualmente al MDP (**0.1 ptos.**).

Es importante recordar que el modelado deberá estar **recogido en la memoria de la práctica** y **no** en un documento a parte. La realización de figuras deberá hacerse empleando herramientas dedicadas y no escaneando trazados/dibujos hechos “a mano”.

4.2. Generación del MDP por iteración de control

Este apartado suma un total de **3 ptos**. Su desglose es:

- Implementar la función que calcula la matriz $\mathcal{P}$. (**0.75 ptos.**)
- Implementar la función que calcula la matriz $\mathcal{C}$ para cada iteración de control. (**1.25 ptos.**)
- Integrar ambas funciones en una que resuelva una iteración de control. (**1 pto.**)

4.3. Integración del bucle de control

Este apartado suma un total de **1.5 ptos**. Su desglose es:

- Implementar la función que realiza todo el bucle de control para generar la respuesta del reactor frente a la curva de demanda de la potencia. (**1.5 ptos.**)

4.4. Implementación de métricas de evaluación

Este apartado suma un total de **0.5 ptos**. Su desglose es:

- Implementar el MAE. (**0.125 ptos.**)
- Implementar el MSE. (**0.125 ptos.**)
- Implementar el R^2. (**0.125 ptos.**)
- Implementar la Correlación de Pearson. (**0.125 ptos.**)

4.5. Experimentación

Este apartado suma un total de **1 pto**. Su desglose es:

- Probar distintos reactores y recoger sus resultados en la memoria. Se valorará positivamente el uso de gráficas y tablas en la misma. (**1 pto.**)

4.6. Memoria

La memoria de la práctica representa un total de **3 ptos**. El objetivo de la memoria es el documentar con detalle la solución que se ha diseñado e implementado, destacando los aspectos técnicos más relevantes. Su desglose es:

- Contenido y explicación del sistema. (**2 ptos.**)
- Estructura y presentación del documento. (**0.5 ptos.**)
- Descripción, justificación y aprendizaje del uso de IA generativa. (**0.5 ptos.**)

Se hace especial hincapié en que el alumno **cuide meticulosamente** la calidad de la memoria, poniendo especial énfasis en las explicaciones claras y formales, así como en el estilo, estética y presentación de la misma. Una buena memoria debería ser capaz de explicar el sistema desarrollado sin necesidad de recurrir prácticamente al código. Esto no signifca que haya que agregar código a la memoria, el alumno **deberá evitarlo** en todo momento. Las memorias deberán tener una **portada**, **índice**, **introducción**, **contenido**, **experimentos** y **conclusiones** como mínimo, **enumerando** las páginas excepto la correspondiente a la portada. Si el alumno recurre a la literatura científica deberá **citar las fuentes** que haya empleado. Si se agregan referencias, se deberá usar el formato de citación `IEEE`.

5. Normas

Esta práctica se debe realizar por grupos de 3 alumnos. La entrega de la práctica consistirá en un fichero comprimido `ZIP` que debe ser enviado por cada grupo a través del entregador proporcionado por Aula Global. El nombre del fichero ZIP debe ser ‘‘`IA_2526_<NIA0>_<NIA1>_<NIA2>.zip`’’, donde `<NIAX>` son los seis últimos dígitos del NIA de cada uno de los miembros del grupo.

- Ejemplo: `IA_2526_123456_345678_456789.zip`

El fichero ZIP incluirá:

- Carpeta con el código correspondiente, separado en ficheros `PY` y `JSON`.
- Memoria técnica de la práctica compilada en formato `PDF`.
- (Opcional) Fichero de texto (`TXT`) README con instrucciones e indicaciones necesarias para facilitar la ejecución del sistema y corrección del mismo, si el alumno lo estima oportuno.

Se habilitará un entregador en Aula Global, plataforma a través de la cual los alumnos deberán subir el fichero `ZIP` descrito anteriormente. La fecha límite para entregar la práctica es el 15 de mayo a las 23:59. No se aceptarán prácticas enviadas por correo electrónico o fuera de la fecha límite de entrega.

6. Uso de IA generativa

Se permite el uso de herramientas de IA generativa, como ChatGPT, siempre que su utilización sea transparente y esté debidamente documentada. En particular, la IA puede emplearse para:

- Buscar información relevante o ejemplos relacionados con la práctica.
- Obtener explicaciones sobre métodos, bibliotecas o conceptos específicos.
- Generar ideas o posibles enfoques para la solución del problema, siempre que el alumno los analice críticamente y los adapte a su propio desarrollo.

Restricciones

- No está permitido utilizar IA generativa para generar código, algoritmos o fragmentos de texto que sean incorporados directamente en el sistema o en la memoria sin modificaciones ni comprensión por parte del alumno.
- Se espera que el alumno diseñe, implemente y documente la solución de manera razonada y justificada.

Declaración de Uso

El alumno deberá incluir un apartado adicional al final de la memoria de la práctica, donde describirá de forma transparente el uso que ha hecho de la IA generativa. En este apartado deberá especificar:

- Para qué aspectos de la práctica ha empleado la IA.
- En qué medida le ha facilitado la realización de la práctica.
- Cómo ha verificado y adaptado la información obtenida.

Esta declaración será evaluada positivamente en la calificación final de la práctica, en función de la calidad y profundidad del análisis realizado.